

Approfondimento sulla sicurezza nelle console di gioco

Marica Bottega – VR519372, Enrico Ciciarella – VR535399

20 maggio 2026

Indice

1	Introduzione	1
2	Memory Corruption	1
3	Protezione della copia e crittografia White-Box	1
4	Fault Injection	3
5	Architetture Moderne	3
6	Fonti	4

1 Introduzione

Lo scopo di questo elaborato è quello di analizzare come le tecniche offensive e difensive nelle console di gioco casalinghe si siano evolute nel tempo.

Il modello di minaccia all'interno del quale sono immersi questi sistemi è il **modello MATE** – *Man At The End*, ovvero quello in cui l'attaccante ha completa disposizione fisica della macchina.

Analizziamo quindi lo stato dell'arte delle tecniche di difesa impiegati dalle più grandi case produttrici di console di gioco, e di come siano comunque state prese di mira attraverso exploit software, corruzione della memoria e attacchi hardware come la *Fault Injection*, con l'obiettivo di aggirare la *Chain of Trust*.

2 Memory Corruption

3 Protezione della copia e crittografia White-Box

Il contesto storico è quello dei primi anni 2000 – periodo in cui erano appena entrate in commercio console come xBox, GameCube e PlayStation 2.

Come anticipato in introduzione, il modello *MATE* assume in partenza che tutto ciò che si trova all'esterno del processore (RAM, bus di sistema, periferiche, ...) sia esposto e potenzialmente sotto il controllo dell'avversario.

Il principio è lo stesso che giustifica la crittografia White-Box: chi esegue il programma è al contempo chi può attaccarlo.

Il contributo di *Weidong Shi, Hsien-Hsin S. Lee, Chenghuai Lu e Tao Zhang*, pur non essendo specifico dell'ambito videoludico, esamina **Attacchi e analisi del rischio per i sistemi hardware a supporto**

della protezione dalla copia del software [1]¹ e cita tra le principali tecniche di attacco hardware per soverchiare le difese software:

- **Trace Analysis:** l'attaccante raccoglie informazioni del software protetto tramite *logging* ed analizzando le tracce d'esecuzione dello stesso. Alcune primitive sono riconoscibili anche dopo essere state cifrate (ad esempio branch condizionali, indici e diversi tipi di variabili).
- **Mod-chip:** con il termine *mod-chip* gli autori si riferiscono a dispositivi designati a mettere alla luce i meccanismi di protezione di copia. Possono anche essere destinati all'appropriazione del segnale di un altro dispositivo o a lanciare attacchi di spoofing ad altri dispositivi.
- **Emulatori:** è complesso combattere contro attacchi di emulazione senza utilizzare la cifratura; tuttavia, non è possibile parlare di protezione della copia del software se il suddetto può essere eseguito su un emulatore senza autorizzazione.

Scendendo ulteriormente nel dettaglio, gli autori analizzano anche altri contesti ed alcuni dei relativi attacchi:

- Analizzando le **lazy authentication**² e le **counter-mode encryption**³, si può parlare ad esempio di **attacco ATT**, ovvero "*alter then trace attack*", il quale prevede che l'attaccante sia in grado di modificare una parte del software (programma o dati) bit-per-bit, che le istruzioni alterate possano essere eseguite e che l'integrità di tali istruzioni/dati non venga verificata tempestivamente né in modo rigoroso istruzione per istruzione.

Il rischio delle autenticazioni "Lazy" può essere ridotto utilizzando *control flow/memory fetch address obfuscation* hardware, nonostante questo possa aumentare notevolmente l'overhead e la complessità dell'hardware stesso.

- Riguardo il **Software slice**, una tecnica che prevede di mettere in sicurezza il software proteggendone solo alcune porzioni, l'attaccante può trattare le porzioni di codice protetto come *black box* e cercare di creare un codice diverso dall'originale ma che riesca ad emettere lo stesso output dallo stesso input – naturalmente si tratta di un problema computazionale complesso se applicato a qualsiasi programma, ma l'avversario potrebbe essere interessato nel corrompere anche solo una *slice* di programma, e potrebbe riuscire a farlo in un tempo ragionevole.
- Gli autori del paper analizzano poi gli **Active Information Flow Attack**. Il contesto prevede che alcuni sistemi utilizzino lo stesso schema di cifratura e la stessa chiave sia per proteggere le istruzioni che i dati dinamici, esponendo il controllo della protezione all'utente tramite istruzioni come **enter security**, **exit security**, **secure load** e **secure store**⁴.

Vengono portati due esempi di attacco a questi sistemi: nel primo, l'attaccante può semplicemente rimpiazzare parti di codice non protetti con contenuto malevolo; dopo che il codice infetto viene criptato, l'attaccante può ridirezionare il puntatore verso tale codice per farlo eseguire in un contesto protetto. Nel secondo esempio, l'attaccante cerca di sfruttare le *linked list* per ridirezionare l'ultimo puntatore della lista ad un elemento segreto per poterlo rivelare in uno spazio non protetto. Le soluzioni a questo tipo di attacchi prevedono anche l'impiego di *program analysis* o di diverse tecniche di compilazione.

¹Titolo originale: *Attacks and Risk Analysis for Hardware Supported Software Copy Protection Systems*

²Con 'lazy' ci si riferisce alla situazione nella quale la sorgente dei dati è utilizzata come un operando ed il risultato della computazione ottenuto utilizzando dati non autenticati è in grado di modificare lo stato del processore prima che l'integrità di tali dati sia confermata.

³La counter-mode encryption (*CTR*), invece di cifrare il messaggio direttamente, si cifra una sequenza di valori numerici incrementali per produrre un flusso di bit pseudo-casuali, detto pad. Il messaggio viene poi combinato con questo pad tramite XOR, ottenendo il ciphertext. Fonte: *Comments to NIST concerning AES Modes of Operations: CTR-Mode Encryption* [2], Lipmaa et al.

⁴Fonte: Table 3, *Attacks and Risk Analysis for Hardware Supported Software Copy Protection Systems* [1], Shi et al.

- In alcuni contesti potrebbe essere necessario utilizzare **cifrature semplici o chiavi corte**, per motivi di performance o di limitazioni hardware. Anche in questo contesto, gli autori presentano due semplici esempi di attacco, uno che mira alla derivazione di un *integrity code* breve a protezione di una porzione di codice tramite *brute force attack*, mentre l'altro che sfrutta le caratteristiche cicliche delle variabili contatore, come il fatto che incrementano in modo regolare, per individuarle (anche se cifrate – ma con una chiave corta) e ricavarne le locazioni in memoria.

4 Fault Injection

5 Architetture Moderne

Riguardo l'analisi delle architetture moderne, ovvero quelle che caratterizzano le console degli anni 2020, la letteratura non offre molti approfondimenti.

I riferimenti di questa sezione provengono richiamano principalmente alla letteratura generica, con un focus particolare sul documento “*Next-Gen Exploitation: Exploring the PS5 Security Landscape*” [3] redatto da SpecterDev, un ricercatore di sicurezza specializzato in kernel exploitation e virtualizzazione, con focus su Android e Linux [4] e che si occupa di sicurezza delle console come attività secondaria dal 2020, con particolare attenzione all'hypervisor di PlayStation 5.

Abbiamo estrapolato dal documento solo alcune delle tecniche difensive adottate da questa nuova console, esposte nei seguenti paragrafi.

XOM e kCFI come ostacolo all'analisi del codice Il kernel implementa anche il software *Control Flow Integrity* (da cui kCFI), che ostacola l'analisi del *Control Flow Graph*.

Al contempo, l'utilizzo della XOM – *eXecute-Only Memory* – mitiga la *Return Oriented Programming* rendendo più complesso trovare i **gadget**, piccoli pezzi di codice che terminano ognuno con un'istruzione **ret** e che, se concatenati, potrebbero portare alla costruzione di operazioni arbitrarie senza mai scrivere nuovo codice. La XOM ostacola sia l'analisi statica (il codice non può essere letto) sia quella dinamica (non si può copiare il binario a *runtime*). I gadget richiedono quindi un attacco **brute force** per essere recuperati.

La XOM, inoltre, non può essere disabilitata con *kernel R/W* arbitrarie poiché le *Nested Page Tables* – che mappano gli indirizzi virtuali a quelli fisici della RAM – non sono accessibili direttamente; la zona proibita è definita dall'**hypervisor** e non dal kernel.

L'hypervisor AMD SVM come environment checker L'hypervisor di PlayStation 5 è un monitor di sicurezza che intercetta le azioni sensibili del kernel guest. Sfrutta l'AMD SVM (*AMD Secure Virtualization*), la tecnologia AMD per la virtualizzazione dell'hardware-backend, per dare vita ad una vera e propria *Virtualization-Based security* per le console.

Ogni volta che il kernel tenta di eseguire una azione sensibile, l'hardware genera un **VMEXIT**: interrompe l'esecuzione e passa il controllo all'hypervisor, che decide se l'operazione è lecita. Si assicura che il kernel non possa disabilitare componenti di sicurezza essenziali (ad esempio modificando i bit relativi a *Write Protect* [WP], *Supervisor Mode Access Protection* [SMAP] o *Supervisor Mode Execution Protection* [SMEP]) o modificando i *Model-Specific Registers* [MSR].

Isolamento crittografico nel co-processore sicuro Su PlayStation 5 le applicazioni e le librerie sono **cifrate su disco**. I file si chiamano SELF e solo i co-processor sicuri (MP0 di Sony e MP3 di AMD PSP) ne possiedono le chiavi. Il kernel x86 fa da intermediario per le richieste di decifratura e verifica, ma non ha mai accesso diretto alle chiavi.

Una grande differenza tra PlayStation 4 e PlayStation 5 è che sulla prima questa API del kernel poteva essere patchata, mentre sulla seconda l'hypervisor lo impedisce.

PS4 vs PS5: evoluzione del modello di post-exploitation In questa ultima sezione, abbiamo voluto evidenziare con un confronto diretto le maggiori differenze a livello di sicurezza tra PlayStation 5 e la sua antecedente PlayStation 4.

Sulla vecchia PlayStation 4 le mitigazioni erano deboli, e spesso una *kernel R/W* era sufficiente – l'assenza di SMAP permetteva di costruire oggetti kernel falsi direttamente nello spazio utente e non c'era nemmeno bisogno di avere a che fare con la *kernel Address Space Layout Randomization* [kASLR], che mescola randomicamente la posizione in memoria del kernel ad ogni avvio, per non rendere noto il luogo in memoria fisso dove si trovano le strutture sensibili.

Su PlayStation 5 gli unici attacchi possibili sono i *data-only attacks* (attacchi che non modificano il flusso di controllo del programma: l'attaccante sfrutta un bug di scrittura in memoria per corrompere dati, non codice [5]), e l'unico modo per accedere senza bug nell'hypervisor e tramite lo spoofing della mailbox PSP (che tuttavia l'autore non ha verificato).

6 Fonti

1. *Attacks and Risk Analysis for Hardware Supported Software Copy Protection Systems*, Shi et al
2. *Comments to NIST concerning AES Modes of Operations: CTR-Mode Encryption*", Lipmaa et al.
3. *Next-Gen Exploitation: Exploring the PS5 Security Landscape*, SpecterDev
4. *Dayzerosec — About SpecterDev*
5. *Data-Only Attacks Are Easier than You Think*, Johannesmeyer et al.